УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность 5-04-0612-02

Учебный предмет Инструментальное программное обеспечение

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____М.М.Федоськова
_____

ЛАБОРАТОРНАЯ РАБОТА № 10

СОЗДАНИЕ И ПОДКЛЮЧЕНИЕ СОБСТВЕННОГО МОДУЛЯ, СОДЕРЖАЩЕГО ПОЛЬЗОВАТЕЛЬСКИЕ ФУНКЦИИ

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал преподаватель
Денисовец Д.А.

2026

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № ____от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Создание и подключение собственного модуля, содержащего пользовательские функции

2 Методическое и материальное обеспечение

2.1 Персональный компьютер
2.2 Методические рекомендации по выполнению лабораторной работы
2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями
3.2 Выполнить индивидуальное задание
3.3 Оформить отчет
3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Модули и пакеты

Модулем в языке **Python** называется любой файл с программным кодом, Каждый модуль может импортировать другой модуль, получая, таким образом, доступ к атрибутам (переменным, функциям и классам), объявленным внутри импортированного модуля. Следует заметить, что импортируемый модуль может содержать программу не только на языке **Python** - так, можно импортировать скомпилированный модуль, написанный на языке **C**.

Все программы, которые запускали ранее, были расположены в модуле с названием **"__main__"**. Получить имя модуля позволяет предопределенный атрибут **__name__**. Для запускаемого модуля он содержит значение **"__main__"**, а для импортируемого модуля - его имя. Выведем название модуля:

```
print (__name__)
```
```
__main__
```

Проверить, является модуль главной программой или импортированным модулем, позволяет код, приведенный ниже:

```
if __name__  ==  "__main__":
    print ("Это главная программа")
else:
    print ("Импортированный модуль")
```

4.2 Инструкция import

Импортировать модуль позволяет инструкция **import**. Ранее уже не раз обращались к этой инструкции для подключения встроенных модулей. Например,

подключали модуль **time** для получения текущей даты с помощью функции **strftime()**:

```
import  time # Импортируем модуль time
print(time.strftime("%d.%m.%Y")) #  Выводим текущую дату
06.02.2018
```

Инструкция **import** имеет следующий формат:

```
import <Название модуля 1>   [as <Псевдоним 1>][, ...,
      <Название модуля N>   [as  <Псевдоним N>]]
```

После ключевого слова **import** указывается название модуля. Обратите внимание на то, что название не должно содержать расширения и пути к файлу. При именовании модулей необходимо учитывать, что операция импорта создает одноименный идентификатор, - это означает, что название модуля должно полностью соответствовать правилам именований переменных. Можно создать модуль с именем, начинающимся с цифры, но подключить такой модуль будет нельзя. Кроме того, следует избегать совпадения имен модулей с ключевыми словами, встроенными идентификаторами и названиями модулей, входящих в стандартную библиотеку.

За один раз можно импортировать сразу несколько модулей, записав их через запятую. Для примера подключим модули **time** и **math**.

```
import time, math # Импортируем несколько модулей сразу
print(time.strftime("%d.%m.%Y")) #  Текущая дата
06.02.2018
print(math.pi) # Число pi
3.141592653589793
```

После импортирования модуля его название становится идентификатором, через который можно получить доступ к атрибутам, определенным внутри модуля. Доступ к атрибутам модуля осуществляется с помощью точечной нотации. Например, обратиться к константе **pi**, расположенной внутри модуля **math**, можно так:

```
math.pi
```

Функция **getattr()** позволяет получить значение атрибута модуля по его названию, заданному в виде строки. С помощью этой функции можно сформировать название атрибута динамически во время выполнения программы. Формат функции:

```
getattr (<Объект молуля>, <Атрибут>[, <Значение по умолчанию>] )
```

Если указанный атрибут не найден, возбуждается исключение **AttributeError**. Чтобы избежать вывода сообщения об ошибке, можно в третьем параметре указать значение, которое будет возвращаться, если атрибут не существует. Пример использования функции приведен ниже.

```
import math
print (getattr (math, "pi")) # Число pi
3.141592653589793
print (getattr(math, "x", 50)) #  Число  50, т. к. x не существует
50
```

Проверить существование атрибута позволяет функция **hasattr (<Объект>, <Название атрибута>)**. Если атрибут существует, функция возвращает значение **True**. Напишем функцию проверки существования атрибута в модуле **math**.

```
import math
def hasattr_math(attr):
    if hasattr(math, attr):
        return "Атрибут существует"
    else:
        return  "Атрибут не существует"
print (hasattr_math("pi")) # Атрибут существует
print(hasattr_math("x")) # Атрибут не существует
```

Если название модуля слишком длинное, и его неудобно указывать каждый раз для доступа к атрибутам модуля, то можно создать псевдоним. Псевдоним задается после ключевого слова **as**. Создадим псевдоним для модуля **math**.

```
import math as m #  Создание псевдонима
print(m.pi)  #  Число pi
3.141592653589793
```

Теперь доступ к атрибутам модуля **math** может осуществляться только с помощью идентификатора **m**. Идентификатор **math** в этом случае использовать уже нельзя.

Все содержимое импортированного модуля доступно только через идентификатор, указанный в инструкции **import**. Это означает, что любая глобальная переменная на самом деле является глобальной переменной модуля. По этой причине модули часто используются как пространства имен. Для примера создадим модуль под названием **tests.py**, в котором определим переменную **x**:

```
# Содержимое модуля tests.py
# -*- coding: utf-8  -*-
x = 50
```

В основной программе также определим переменную **x**, но с другим значением. Затем подключим файл **tests.py** и выведем значения переменных:

```
# Содержимое основной программы
# -*- coding: utf-8  -*-
import tests        # Подключаем файл tests,py
x = 22
print (tests.x) # Значение переменной x внутри модуля
print(x)        # Значение переменной x в основной программе
input()
```

Оба файла размещаем в одной папке, а затем запускаем файл с основной программой с помощью двойного щелчка на значке файла (рисунок 1).

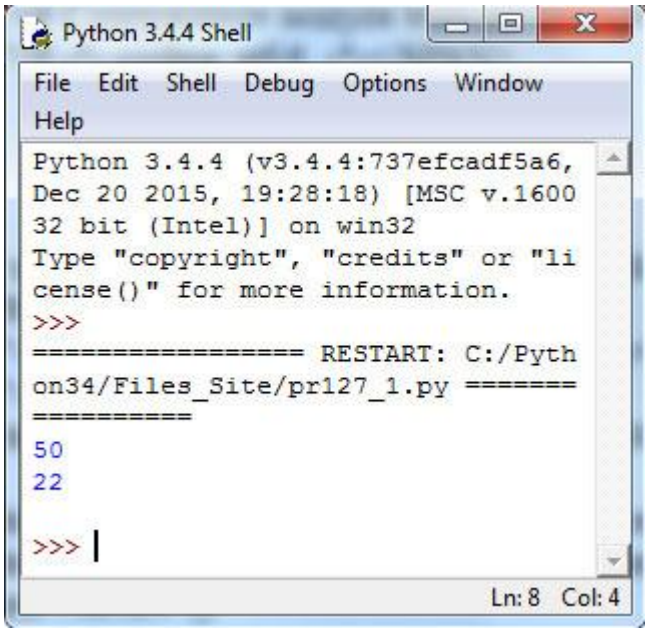


Рисунок 1 - Результат работы приложения

Как видно из результата, никакого конфликта имен нет, поскольку одноименные переменные расположены в разных пространствах имен.

Как говорилось на шаге 2, перед собственно выполнением каждый модуль **Python** компилируется, преобразуясь в особое внутреннее представление (байт-код), - это делается для ускорения выполнения кода. Файлы с откомпилированным кодом хранятся в папке **__pycache__**, автоматически создающейся в папке, где находится сам файл с исходным, неоткомпилированным кодом модуля, и имеют имена вида **<*имя файла с исходным кодом*>.cpython-<*первые две цифры номера версии* Python>.pyc**. Так, при запуске на исполнение нашего файла **tests.py** откомпилированный код будет сохранен в файле **tests.cpython-34.pyc**.

Следует заметить, что для импортирования модуля достаточно иметь только файл с откомпилированным кодом, файл с исходным кодом в этом случае не нужен. Для примера переименуйте файл **tests.py** (например, в **tests1.py**), скопируйте файл **tests.cpython-34.pyc** из папки **__pycache__** в папку с основной программой и переименуйте его в **tests.pyc**, а затем запустите основную программу. Программа будет нормально выполняться. Таким образом, чтобы скрыть исходный код модулей, можно поставлять программу клиентам только с файлами, имеющими расширение **pyc**.

Существует еще одно обстоятельство, на которое следует обратить особое внимание. Импортирование модуля выполняется только при первом вызове инструкции **import** (или **from**, речь о которой пойдет в следующем шаге). При каждом вызове инструкции **import** проверяется наличие объекта модуля в словаре **modules** из модуля **sys**. Если ссылка на модуль находится в этом словаре, то модуль повторно импортироваться не будет. Для примера выведем ключи словаря **modules**, предварительно отсортировав их.

```
# Вывод ключей словаря modules
```

```
# -*- coding: utf-8  -*-
import tests, sys # Подключаем модули tests и sys
print(sorted(sys.modules.keys()))
input()
```

Результат работы приложения изображен на рисунке 2.



Рисунок 2 - Результат работы приложения

Инструкция **import** требует явного указания объекта модуля. Так, нельзя передать название модуля в виде строки. Чтобы подключить модуль, название которого создается динамически в зависимости от определенных условий, следует воспользоваться функцией **__import__()**. Для примера подключим модуль **tests.py** с помощью функции **__import__()**.

```
# Использование функции __import__()
# -*- coding: utf-8  -*-
s = "test" + "s" # Динамическое создание названия модуля
m = __import__(s) # Подключение модуля tests
print(m.x) # Вывод значения атрибута x. Выведет: 50
input ()
```

Получить список всех идентификаторов внутри модуля позволяет функция **dir()**. Кроме того, можно воспользоваться словарем **__dict__**, который содержит все идентификаторы и их значения.

```
# Вывод списка всех идентификаторов
# -*- coding: utf-8  -*-
import tests
print(dir(tests))
print(sorted(tests.__dict__.keys() ))
input ()
```

Результат работы приложения:

```
['__builtins__', '__cached__', '__doc__', '__file__', '__loader__',
'__name__', '__package__', '__spec__', 'x']
['__builtins__', '__cached__', '__doc__', '__file__', '__loader__',
'__name__', '__package__', '__spec__', 'x']
```

4.3 Инструкция from

Для импортирования только определенных идентификаторов из модуля можно воспользоваться инструкцией **from**. Ее формат таков:

```
from <Название модуля> import  <Идентификатор 1> [as <Псевдоним 1>]
            [, ...,] <Идентификатор N> [as  <Псевдоним N>] ]
from <Название модуля> import (<Идентификатор 1> [as <Псевдоним 1>]
            [, ...,  <Идентификатор N> [as <Псевдоним N>]] )
from <Название модуля> import *
```

Первые два варианта позволяют импортировать модуль и сделать доступными только указанные идентификаторы. Для длинных имен можно назначить псевдонимы, указав их после ключевого слова **as**. В качестве примера сделаем доступными константу **pi** и функцию **floor ()** из модуля **math**, а для названия функции создадим псевдоним:

```
# -*- coding: utf-8  -*-
from math import pi, floor as f
print(pi) # Вывод числа pi
# Вызываем функцию floor() через идентификатор f
print(f(5.49)) # Выведет: 5
```

```
input()
```

Результат:

```
3.141592653589793
5
```

Идентификаторы можно разместить на нескольких строках, указав их названия через запятую внутри круглых скобок:

```
from math  import (pi, floor, sin, cos)
```

Третий вариант формата инструкции **from** позволяет импортировать из модуля все идентификаторы. Для примера импортируем все идентификаторы из модуля **math**.

```
# -*- coding: utf-8  -*-
from math import * # Импортируем все идентификаторы из модуля math
print(pi) # Вывод числа pi
print(floor(5.49)) # Выведет: 5
input()
```

Результат:

```
3.141592653589793
5
```

Следует заметить, что идентификаторы, названия которых начинаются с символа подчеркивания, импортированы не будут. Кроме того, необходимо учитывать, что импортирование всех идентификаторов из модуля может нарушить пространство имен главной программы, т. к. идентификаторы, имеющие одинаковые имена, будут перезаписаны. Создадим два модуля и подключим их с помощью инструкций **from** и **import**. Содержимое файла **module1.py** приведено ниже:

```
# Содержимое файла module1.py
# -*- coding: utf-8 -*-
s = "Значение из модуля module1"
```

Содержимое файла **module2.py**:

```
# Содержимое файла module2.py
# -*- coding: utf-8 -*-
s = "Значение из модуля module2"
```

Исходный код основной программы:

```
# Исходный код основной программы
# -*- coding: utf-8  -*-
from module1 import *
from module2 import *
import module1, module2
print(s) # Выведет: "Значение из модуля module2"
print(module1.s) # Выведет: "Значение из модуля modulel"
print(module2.s) # Выведет: "Значение из модуля module2"
input()
```

Результат:

```
Значение из модуля module2
Значение из модуля module1
```

```
Значение из модуля module2
```

Итак, в обоих модулях определена переменная с именем **s**. Размещаем все файлы в одной папке, а затем запускаем основную программу с помощью двойного щелчка на значке файла. При импортировании всех идентификаторов значением переменной **s** будет значение из модуля, который был импортирован последним, - в нашем случае это значение из модуля **module2.py**. Получить доступ к обеим переменным можно только при использовании инструкции **import**. Благодаря точечной нотации пространство имен не нарушается.

В атрибуте **__all__** можно указать список идентификаторов, которые будут импортировать ся с помощью выражения **from module import ***. Идентификаторы внутри списка указываются в виде строки. Создадим файл **module3.py**:

```
# Содержимое файла module3.py
# Использование атрибута __all__
# -*- coding: utf-8  -*-
x, y, z, _s = 10, 80, 22, "Строка"
__all__ = ["x", "_s"]
```

Затем подключим его к основной программе:

```
# Исходный код основной программы
# -*- coding: utf-8  -*-
from module3 import *
print(sorted(vars().keys())) # Получаем список всех идентификаторов
input()
```

После запуска основной программы (с помощью двойного щелчка на значке файла) получим следующий результат:

```
['__builtins__', '__doc__', '__file__', '__loader__', '__name__',
'__package__', '__spec__', '_s', 'x']
```

Как видно из примера, были импортированы только переменные **_s** и **x**. Если не указать идентификаторы внутри списка **__all__**, то результат будет другим:

```
['__builtins__', '__doc__', '__file__', '__loader__', '__name__',
'__package__', '__spec__', 'x', 'y', 'z']
```

Обратите внимание на то, что переменная **_s** в этом случае не импортируется, т. к. ее имя начинается с символа подчеркивания.

4.4 Пути поиска модулей

До сих пор мы размещали модули в одной папке с файлом основной программы. В этом случае нет необходимости настраивать пути поиска модулей, т. к. папка с исполняемым файлом автоматически добавляется в начало списка путей. Получить полный список путей поиска позволяет следующий код:

```
>>> import  sys #  Подключаем модуль  sys
>>> sys.path # path содержит список путей поиска модулей
```

Список **sys.path** содержит пути поиска, получаемые из следующих источников:

— путь к текущему каталогу с кодом основной программы;

— значение переменной окружения **PYTHONPATH**. Для добавления переменной в меню ***Пуск*** выбираем пункт ***Панель управления*** (или ***Настройка | Панель управления***). В открывшемся окне выбираем пункт ***Система*** и, если мы пользуемся **Windows Vista** или более новой версией этой системы, щелкаем на ссылке ***Дополнительные параметры системы***. Переходим на вкладку ***Дополнительно*** и нажимаем кнопку ***Переменные среды***. В разделе ***Переменные среды*** пользователя нажимаем кнопку ***Создать***. В поле ***Имя переменной*** вводим **PYTHONPATH**, а в поле ***Значение переменной*** задаем пути к папкам с модулями через точку с запятой — например, **C:\folder1;C:\folder2**. Закончив, не забудем нажать кнопки **ОК** обоих открытых окон. После этого изменения перезагружать компьютер не нужно, достаточно заново запустить программу;

— пути поиска стандартных модулей;

— содержимое файлов с расширением **pth**, расположенных в каталогах поиска стандартныхмодулей, — например, в каталоге **C:\Python34\Lib\site-packages**. Названия таких файлов могут быть произвольными, главное, чтобы они имели расширение **pth**. Каждый путь (абсолютный или относительный) должен быть расположен на отдельной строке. Для примера создайте файл **mypath.pth** в каталоге **C:\Python34\Lib\site-packages** со следующим содержимым:

```
# Это комментарий
C:\folder1
C:\folder2
```

При поиске модуля список **sys.path** просматривается слева направо. Поиск прекращается после первого найденного модуля. Таким образом, если в каталогах **C:\folder1** и **C:\folder2** существуют одноименные модули, то будет использоваться модуль из папки **C:\folder1**, т. к. он расположен первым в списке путей поиска.

Список **sys.path** можно изменять из программы с помощью списковых методов. Например, добавить каталог в конец списка можно с помощью метода **append()**, а в его начало — с помощью метода **inser ()**:

```
# Изменение списка путей поиска модулей
# -*-  coding: utf-8 -*-
import sys
sys.path.append(r"C:\folder1") # Добавляем в  конец списка
sys.path.insert(0,r"C:\folder2") # Добавляем в  начало списка
print(sys.path)
input ()
```

Результат работы приложения:

```
['C:\\folder2', 'C:\\Python34\\Lib\\idlelib', 'C:\\Python34\\python34.zip',
'C:\\Python34\\DLLs', 'C:\\Python34\\lib', 'C:\\Python34',
'C:\\Python34\\lib\\site-packages', 'C:\\folder1']
```

В этом примере мы добавили папку **C:\folder2** в начало списка. Теперь, если в каталогах **C:\folder1** и **C:\folder2** существуют одноименные модули, будет использоваться модуль из папки **C:\folder2**, а не из папки **C:\folder1**, как в предыдущем примере.

Обратите внимание на символ **r** перед открывающей кавычкой. В этом режиме специальные последовательности символов не интерпретируются. Если используются обычные строки, то в указании пути необходимо удвоить (экранировать) каждый встреченный слеш:

```
sys.path.append("C:\\folder1\\folder2\\folder3")
```

4.5 Повторная загрузка модулей

Как вы уже знаете, модуль загружается только один раз при первой операции импорта. Все последующие операции импортирования этого модуля будут возвращать уже загруженный объект модуля, даже если сам модуль был изменен. Чтобы повторно загрузить модуль, следует воспользоваться функцией **reload()** из модуля **imp**. Формат функции:

```
from imp import reload
reload(<Объект модуля>)
```

В качестве примера создадим модуль **tests2.py** со следующим содержимым:

```
# -*-  coding: utf-8 -*-
x = 150
```

Подключим этот модуль в окне **Python Shell** редактора **IDLE** и выведем текущее значение переменной **x**:

```
>>> import tests2 # Подключаем модуль tests2.py
>>> print(tests2.x) # Выводим текущее значение
150
```

Не закрывая окно **Python Shell**, изменим в модуле значение переменной **x** на 800, а затем попробуем заново импортировать модуль и вывести текущее значение переменной:

```
>>> # Изменяем значение в модуле на  800
>>> import tests2
>>> print (tests2.x) # Значение  не изменилось
150
```

Как видно из примера, значение переменной **x** не изменилось. Теперь перезагрузим модуль с помощью функции reload():

```
>>> from imp import reload
>>> reload (tests2) #  Перезагружаем модуль
<module 'tests2' from 'C:/Python34/files\\tests2.py'>
>>> print(tests2.x) # Значение изменилось
800
```

При использовании функции **reload()** следует учитывать, что идентификаторы, импортированные с помощью инструкции **from**, перезагружены не будут. Кроме того, повторно не загружаются скомпилированные модули, написанные на других языках программирования, — например, на языке **C**.

4.6 Пакеты

Пакетом называется каталог с модулями, в котором расположен файл инициализации **__init__.py**. Файл инициализации может быть пустым или содержать код, который будет выполнен при первом обращении к пакету. В любом случае он обязательно должен присутствовать внутри каталога с модулями.

В качестве примера создадим следующую структуру файлов и каталогов:

```
main.py              # Основной файл с программой
folder1\             # Папка  на  одном уровне вложенности с main.py
   __init__.py       # Файл инициализации
   module1.py        # Модуль   folder1\module1.py
   folder2\          # Вложенная папка
      __init__.py    # Файл инициализации
      module2.py     # Модуль  folder1\folder2\module2.py
      module3.py     # Модуль  folder1\folder2\module3.py
```

Содержимое файлов **__init__.py**:

```
# -*- coding: utf-8 -*-
print ("__init__ из",__name__)
```

Содержимое модулей **module1.py**, **module2.py** и **module3.py**:

```
# -*- coding: utf-8  -*-
msg = "Модуль {0}".format(__name__)
```

Теперь импортируем эти модули в основном файле **main.py** и получим значение переменной **msg** разными способами. Файл **main.py** будем запускать с помощью двойного щелчка на значке файла.

Содержимое файла **main.py**:

```
#-*- coding: utf-8 -*-
# Доступ к модулю folderl\modulel.py
import folder1.module1 as m1
            # Выведет: __init__ из folder1
print(m1.msg)    # Выведет: Модуль folder1.module1
from folder1 import module1 as m2
print(m2.msg)    # Выведет: Модуль folder1.module1
from folder1.module1 import msg
print(msg)       # Выведет: Модуль folder1.module1
# Доступ к модулю folder1\folder2\module2.py
import folder1.folder2.module2 as m3
# Выведет: __init__ из folderl.folder2
print(m3.msg)    # Выведет: Модуль folder1.folder2.module2
from folder1.folder2 import module2 as m4
print(m4.msg)    # Выведет: Модуль folder1.folder2.module2
from folder1.folder2.module2 import msg
print(msg)     # Выведет: Модуль folder1.folder2.module2
input ()
```

Как видно из примера, пакеты позволяют распределить модули по каталогам. Чтобы импортировать модуль, расположенный во вложенном каталоге,

необходимо указать путь к нему, перечислив имена каталогов через точку. Если модуль расположен в каталоге **C:\folder1\folder2**, то путь к нему из **C:** должен быть записан так: **folder1.foider2**. При использовании инструкции **import** путь к модулю должен включать не только названия каталогов, но и название модуля без расширения:

```
import folder1.folder2.module2
```

Получить доступ к идентификаторам внутри импортированного модуля можно следующим образом:

```
print(folder1.folder2.module2.msg)
```

Так как постоянно указывать столь длинный идентификатор очень неудобно, можно создать **псевдоним**, указав его после ключевого слова as, и обращаться к идентификаторам модуля через него:

```
import  folder1.folder2.module2  as m
print(m.msg)
```

При использовании инструкции **from** можно импортировать как объект модуля, так и определенные идентификаторы из модуля. Чтобы импортировать объект модуля, его название следует указать после ключевого слова **import**:

```
from folder1.folder2  import module2
print(module2.msg)
```

Для импортирования только определенных идентификаторов название модуля указывается в составе пути, а после ключевого слова **import** через запятую перечисляются идентификаторы:

```
from folder1.folder2.module2 import msg
print (msg)
```

Если необходимо импортировать все идентификаторы из модуля, то после ключевого слова **import** указывается символ *:

```
from folder1.folder2.module2 import *
print (msg)
```

Инструкция **from** позволяет также импортировать сразу несколько модулей из пакета. Для этого внутри файла инициализации **__init__.py** в атрибуте **__all__** необходимо указать список модулей, которые будут импортироваться с помощью выражения **from пакет import ***.

В качестве примера изменим содержимое файла **__init__.py** из каталога **folder1\folder2**:

```
# -*- coding: utf-8 -*-
# print ("__init__ из",__name__)
__all__ = ["module2", "module3"]
```

Теперь изменим содержимое основного файла **main.py** и запустим его.

```
#-*- coding: utf-8 -*-
from folder1.folder2 import *
print(module2.msg)       # Выведет: Модуль folder1.folder2.module2
print(module3.msg)       # Выведет: Модуль folder1.folder2.module3
input ()
```

Как видно из примера, после ключевого слова **from** указывается лишь путь к каталогу без имени модуля. В результате выполнения инструкции **from** все

модули, указанные в списке **__all__**, будут импортированы в пространство имен модуля **main.py**.

До сих пор мы рассматривали импортирование модулей из основной программы. Теперь рассмотрим импорт модулей внутри пакета. В этом случае инструкция **from** поддерживает относительный импорт модулей. Чтобы импортировать модуль, расположенный в том же каталоге, перед названием модуля указывается точка:

```
from .module import *
```

Чтобы импортировать модуль, расположенный в родительском каталоге, перед названием модуля указываются две точки:

```
from ..module import *
```

Если необходимо обратиться еще уровнем выше, то указываются три точки:

```
from ...module import *
```

Чем выше уровень, тем больше точек необходимо указать. После ключевого слова **from** можно указывать одни только точки - в этом случае имя модуля вводится после ключевого слова **import**. Пример:

```
from .. import module
```

Рассмотрим относительный импорт на примере. Для этого изменим содержимое модуля **module3.py**:

```
# -*- coding: utf-8 -*-
# Импорт модуля module2.py из текущего каталога
from . import module2 as m1
var1 = "Значение из: {0}".format(m1.msg)
from .module2 import msg as m2
var2 = "Значение из: {0}".format(m2)
# Импорт модуля module1.py из родительского каталога
from .. import module1 as m3
var3 = "Значение из: {0}".format(m3.msg)
from ..module1 import msg as m4
var4 = "Значение из: {0}".format(m4)
```

Теперь изменим содержимое основного файла **main.py** и запустим его с помощью двойного щелчка на значке файла.

```
#-*- coding: utf-8 -*-
from folder1.folder2 import module3 as m
print(m.var1) # Значение из: Модуль folder1.folder2.module2
print(m.var2) # Значение из: Модуль folder1.folder2.module2
print(m.var3) # Значение из: Модуль folder1.modulel
print(m.var4) # Значение из: Модуль folder1.modulel
input ()
```

При импортировании модуля внутри пакета с помощью инструкции **import** важно помнить, что в **Python 3** производится абсолютный импорт. Если при запуске **Python**-программы с помощью двойного щелчка на ее файле автоматически добавляется путь к каталогу с исполняемым файлом, то при импорте внутри пакета этого не происходит. Поэтому если изменить содержимое

модуля **module3.py** показанным далее способом, то мы получим сообщение об ошибке или загрузим совсем другой модуль:

```
#-*- coding: utf-8 -*-
import module2 #  Ошибка! Поиск модуля по абсолютному пути
var1  = "Значение из: {0}".format(module2.msg)
var2 = var3 = var4  = 0
```

В этом примере мы попытались импортировать модуль **module2.py** из модуля **module3.py**. При этом с помощью двойного щелчка мы запускаем файл **main.py** (последний). Поскольку импорт внутри пакета выполняется по абсолютному пути, поиск модуля **module2.py** не будет производиться в папке **folder1\folder2**. В результате модуль не будет найден. Если в путях поиска модулей находится модуль с таким же именем, то будет импортирован модуль, который мы и не предполагали подключать.

Чтобы подключить модуль, расположенный в той же папке внутри пакета, необходимо воспользоваться относительным импортом с помощью инструкции **from**:

```
from . import module2
```

Или указать полный путь относительно корневого каталога пакета:

```
import  folder1.folder2.module2  as module2
```

4.7 Пример решения задачи на языке программирования Python

Задача. Создайте модуль с именем modul_ctg.py, в котором будет находиться функция для вычисления значения функции ctg(x). Подключите модуль в основной программе и выполните несколько вычислений.

Встроенная функция нахождения котангенса числа отсутствует в библиотеке математических функций языка Python.

Ниже представлен программный код, в котором написана функция ctg**,** а также осуществляется ее вызов несколько раз.

Решение задачи:

```
from math import * #Подключение библиотеки математических функций
def ctg(chislo):
   ctg=cos(chislo)/sin(chislo)
   return ctg
x=float(input("\nВведите первое число "))
y=float(input("\nВведите второе число "))
rez=ctg(x)
rez1=ctg(y)
print("\nПервый вызов функции ctg =", rez)
print("\nВторой вызов функции ctg =", rez1)
```

Убедившись в работоспособности программы, скопируем код функции и, создав новый файл, вставим ее текст в него.

Выполним команду **File/Save** и сохраним код в текущем каталоге. Лучше всего, если это будет тот каталог, где вы сохраняете программы, написанные на

Python. В данном случае файл был сохранен под именем **modul_ctg.py.** Созданный файл можно закрыть.

Итогом стало создание **модуля,** который представляет собой обычную программу, написанную на Python. Теперь представим, что мы не предпринимали никаких шагов по созданию программы, а нам сообщили: чтобы воспользоваться вычислением котангенса какого-либо числа, вам нужно подключить в будущей программе модуль, который имеет название **modul_ctg.**

Каким же будет алгоритм использования функции **ctg(x)** в программе? Очень простым. Он будет аналогичен вызову обычной математической функции языка Python.

```
import modul_ctg
x=float(input("\nВведите первое число "))
y=float(input("\nВведите второе число "))
rez=modul_ctg.ctg(x)
rez1=modul_ctg.ctg(y)
print("\nПервый вызов функции ctg =", rez)
print("\nВторой вызов функции ctg =", rez1)
```

В первой строке программного кода с помощью инструкции **import** мы подключили модуль с именем **modul_ctg.** Ранее эта инструкция использовалась нами для подключения стандартных математических функций (**import math**) или для генерации случайных чисел (**import random**). Далее указываем имя модуля, ставим точку и дописываем имя функции. Итак, очевидно, что процесс создания модуля в Python и вызов из него функции не требует использования сложных алгоритмов и большого промежутка времени для реализации задуманного.

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Создайте модуль с именем calc.py, в котором будут находиться функции для выполнения сложения, вычитания, умножения и деления. Подключите модуль в основной программе и выполните несколько вычислений.

2 Создайте модуль geometry.py, содержащий функции вычисления площади круга, квадрата и прямоугольника. Подключите модуль в основном файле и протестируйте работу функций.

3 Создайте модуль strings.py, в котором будет функция подсчёта количества гласных букв в переданной строке. Используйте модуль в основной программе.

4 Создайте модуль converter.py с функциями преобразования значений: километры в мили, градусы Цельсия в градусы Фаренгейта, секунды в минуты. Подключите модуль и проверьте работу каждой функции.

5 Создайте модуль arrays.py, содержащий функции поиска максимального и минимального значений в списке. Используйте модуль в основной программе.

6 Создайте модуль texttools.py, в котором реализуйте функцию удаления всех пробелов из строки. В основной программе вызовите эту функцию и проверьте её работу.

7 Создайте модуль factorial.py, содержащий две функции вычисления факториала: рекурсивную и итерационную. Подключите модуль и сравните работу функций.

8 Создайте модуль stats.py, в котором будут функции вычисления среднего арифметического, медианы и моды списка. Используйте модуль в основной программе.

9 Создайте модуль password.py, в котором будет функция генерации случайного пароля заданной длины. Примените эту функцию в основной программе.

10 Создайте модуль mathseq.py с функциями генерации чисел Фибоначчи и простых чисел, подключите модуль и выведите результаты.

11 Создайте модуль dateutils.py, содержащий функцию, определяющую, является ли год високосным. В основном файле вызовите функцию для нескольких значений.

12 Создайте модуль validator.py, содержащий функции проверки правильности email-адреса, номера телефона и почтового индекса. Используйте модуль в основной программе.

13 Создайте модуль randomlist.py, который содержит функцию генерации списка случайных чисел заданной длины. Импортируйте этот модуль и используйте список в основной программе.

14 Создайте модуль matrix.py, содержащий функции сложения и умножения матриц. Подключите модуль в основной программе и протестируйте его работу.

15 Создайте модуль sortutils.py, содержащий функции сортировки списка методами пузырька и вставок. Импортируйте модуль в основной программе и сравните результаты работы функций.

5.2 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Создайте модуль logic.py, содержащий функции проверки чётности числа, проверки простого числа и проверки делимости. Используйте модуль в основной программе.

2 Создайте модуль matrix.py с функциями сложения двух матриц, умножения матрицы на число и нахождения транспонированной матрицы. Подключите и протестируйте.

3 Создайте модуль strings_extra.py, содержащий функции подсчёта согласных букв и функции удаления цифр из строки. Проверьте работу функций в основном файле.

4 Создайте модуль physics.py с функциями вычисления скорости, пути и ускорения по формулам школьного курса. Подключите модуль и выполните несколько расчётов.

5 Создайте модуль tempconv.py, содержащий функции перевода температур: Фаренгейт → Цельсий, Кельвин → Цельсий, Цельсий → Кельвин. Проверьте работу модуля.

6 Создайте модуль sums.py, содержащий функции суммы первых N чисел, суммы чётных чисел и суммы нечётных чисел. Импортируйте модуль и протестируйте.

7 Создайте модуль textstats.py, содержащий функции подсчёта количества слов, предложений и символов в строке. Используйте в основной программе.

8 Создайте модуль listops.py, содержащий функции удаления дубликатов, сортировки по возрастанию и переворота списка. Примените модуль в основном файле.

9 Создайте модуль password_check.py с функцией проверки сложности пароля (длина, цифры, буквы, спецсимволы). Подключите модуль и протестируйте.

10 Создайте модуль games.py с функциями генерации случайного хода (камень/ножницы/бумага), случайного числа от 1 до 100 и функции проверки выигрыша. Проверьте модуль.

11 Создайте модуль sequences.py с функциями генерации арифметической и геометрической прогрессий. Подключите и используйте в основном файле.

12 Создайте модуль fibtools.py, содержащий функции определения, является ли число числом Фибоначчи, и нахождения ближайшего числа Фибоначчи. Импортируйте модуль и протестируйте.

13 Создайте модуль shapes.py, содержащий функции вычисления периметра треугольника, прямоугольника и окружности. Используйте модуль в основной программе.

14 Создайте модуль files_utils.py, содержащий функции чтения текста из файла, записи текста в файл и подсчёта количества строк в файле. Подключите модуль и протестируйте его работу.

15 Создайте модуль data_analysis.py, содержащий функции поиска максимального и минимального значения в списке, вычисления среднего значения и определения количества положительных элементов. Импортируйте модуль и проверьте его работу в основной программе.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы

6.2 Цель лабораторной работы

6.3 Индивидуальное задание (текст программы и скриншоты выполнения)

7 Контрольные вопросы

7.1 Дайте определение понятию «Модуль» в языке программирования Python.

7.2 Дайте определение понятию «Пакет» в языке программирования Python.

7.3 Опишите процесс импортирования модуля в программу в языке программирования Python?

7.4 Опишите конструкцию import module as m в языке программирования Python?

7.5 Охарактеризуйте условие if __name__ == "__main__": в языке программирования Python?

Список используемых источников

1 Постолит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.

2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.

3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.